

ПРОТОКОЛ ИНФОРМАЦИОННОГО ВЗАИМОДЕЙСТВИЯ ГОЭН-60 С ВЫЧИСЛИТЕЛЕМ ПО ИНТЕРФЕЙСУ RS485

Версия документа: 1.0
Дата: 18 апреля 2025 г.

1. Введение

Данный документ описывает протокол информационного взаимодействия между ГОЭН-60 (далее — **устройство**) и управляющим вычислителем (далее — **хост**) через интерфейс RS485. Протокол основан на спецификации VDB версии 1.0 (см. документ "Протокол VDB ") и предназначен для управления режимами работы устройства, его позицией, скоростью, а также для получения телеметрии.

2. Подключение по интерфейсу RS485

Режим работы устройства: **Slave**.

Параметры передачи:

Скорость (<i>baudRate</i>):	115200 бит/с.
Биты данных (<i>dataBits</i>):	8
Четность (<i>parity</i>):	NoParity.
Стоп-биты (<i>stopBits</i>):	OneStop

3. Формат сообщений

Каждое сообщение имеет следующий формат:

[Стартовый байт] [VDB сообщение] [CRC8]

, где:

- **Стартовый байт:** 0xAA.
- **VDB сообщение:** Сообщение в формате VDB (см. спецификацию VDB).
- **CRC8:** Контрольная сумма, рассчитанная с параметрами:
 - Полином: 0x31.
 - Начальное значение: 0xFF.
 - XOR-значение: 0x00.
- **Адрес устройства для протокола VDB:** 0x01
- **Адрес хоста для протокола VDB:** 0x00

Примечание: Все данные в сообщениях передаются в порядке **little-endian**.

4. Команды устройства

Устройство поддерживает команды, каждая из которых имеет функциональный адрес (*dst.function*). Команды делятся на запросы записи и чтения.

4.1. Таблица команд

Команда	Адрес (<i>dst.function</i>)	Тип запроса	Размер данных (<i>byteCount</i>), байт	Диапазон данных	Описание
<i>mode</i>	0x01	w	1	(uint8_t) 0xC0–0xC4	Установка режима работы
<i>controlPosition</i>	0x02	w	8	Угловое положение по оси курс (float32): -180.0 ... +180.0 град	Управление по положению
				Угловое положение по оси тангаж (float32): -180.0 ... +180.0 град	
<i>controlSpeed</i>	0x03	w	8	Курс (float32): -180.0 ... +180.0 град/с	Управление по скорости
				Тангаж (float32): -180.0 ... +180.0 град/с	
<i>controlTracking</i>	0x04	w	16	Рассогласование по оси курса (float32): -45.0 ... +45.0 град	Управление относительными углами
				Рассогласование по оси тангажа (float32): -45.0 ... +45.0 град	
				Ограничение ускорения по оси курса (float32): 0.0 ... 180.0 град/с ²	
				Ограничение ускорения по оси тангажа (float32): 0.0 ... 180.0 град/с ²	
<i>status</i>	0x05	r	47	—	Состояние устройства (см. 5.1 Структура TelemetryUART)
<i>version</i>	0x00	r	12	—	Версии протокола и устройства (см 6.3. Структура Version)

4.2. Разрешенные команды по режимам

Устройство поддерживает следующие режимы работы, в каждом из которых доступны определенные команды:

Режим *stabilization* (0xC1): *mode, controlPosition, controlSpeed, controlTracking, status.*

Режим *rotaryPlatform* (0xC2): *mode, controlPosition, controlSpeed, status.*

Режим *parkin* (0xC3): *mode, status.*

Режим *off* (0xC4): *mode, status.*

Режим *dummy* (0xC0): *mode, status* (команды записи игнорируются).

Примечание: В режиме *dummy* устройство находится в переходном состоянии (не более 5 секунд), команды записи не выполняются.

5. Телеметрия

Телеметрия возвращается в ответ на команду *status* в структуре *TelemetryUART* размером 47 байт. Данные передаются в порядке little-endian.

5.1. Структура *TelemetryUART*

Поле	Тип	Размер (байт)	Описание
mode	uint8_t	1	Текущий режим устройства (0xC0–0xC4)
mems	MemsBoardStatus	24	Статус MEMS платы
motorYaw	MotorStatus	11	Статус мотора курса
motorPitch	MotorStatus	11	Статус мотора тангажа

Всего: $1 + 24 + 11 + 11 = 47$ байт.

5.2. Структура *MemsBoardStatus*

Поле	Тип	Размер (байт)	Описание
mode	uint8_t	1	Режим стабилизатора
errors	MemsErrors	1	Ошибки MEMS (байт с битовыми полями)
flags	MemsFlags	4	Флаги MEMS (4 байта с битовыми полями)
AxisSwitchMode	uint8_t	1	Режим переключения осей
MemsProcessMode	uint8_t	1	Режим обработки данных MEMS
speedYaw	float32	4	Скорость по оси курса (град/с)
speedPitch	float32	4	Скорость по оси тангажа (град/с)
angleYaw	float32	4	Угол по оси курса (град)
anglePitch	float32	4	Угол по оси тангажа (град)

Всего: $1 + 1 + 4 + 1 + 1 + 4 + 4 + 4 + 4 = 24$ байта.

5.3. Структура `MemsErrors`

Бит **Описание**

Бит	Описание
0	ошибка всех MEMS
1–7	Зарезервировано

5.4. Структура MemsFlags

Бит	Описание
0	битовые флаги для MEMS 1–8 (1 = не настроен)
1	битовые флаги для MEMS 1–8 (1 = неверные данные)
2	битовые флаги для MEMS 1–8 (1 = нет ответа по SPI)
3	битовые флаги

5.5. Структура MotorStatus

Поле	Тип	Размер (байт)	Описание
mode	uint8_t	1	Режим мотора
errors	MotorErrors	1	Ошибки мотора (байт с битовыми полями)
flags	MotorFlags	1	Флаги мотора (байт с битовыми полями)
angle	float32	4	Угол мотора (град)
speed	float32	4	Скорость мотора (град/с)

Всего: $1 + 1 + 1 + 4 + 4 = 11$ байт.

5.6. Структура MotorErrors

Бит	Описание
0	перегрузка по току
1	перегрев
2	превышение скорости
3	слабое магнитное поле
4	низкое напряжение
5-7	Зарезервировано

5.7. Структура MotorFlags

Бит	Описание
0	ошибка
1	ограничение сектора включено
2	калибровка включена
3	несохраненные данные
4-7	Зарезервировано

6. Версия протокола

6.1. Запрос версии

[VDB: cmd=0x01, status=0x80, src=0x0000, dst=0x0010, byteCount=0]

6.2. Ответ на запрос версии

[VDB: cmd=0x81, status=0x80, src=0x0010, dst=0x0000, byteCount=12, payload = Version]

6.3. Структура Version

Поле	Тип	Размер (байт)	Описание
versionProtocolVDB	uint32_t	4	Версия протокола VDB (например, 1.0)
versionProtocol	uint32_t	4	Версия протокола взаимодействия
versionDevice	uint32_t	4	Версия устройства

Всего: 4 + 4 + 4 = 12 байт.

7. Обработка ошибок

При возникновении ошибки устройство возвращает сообщение с кодом ошибки.

7.1. Формат сообщения с ошибкой

- Для команд записи: $cmd = 0x80$, $status = 0x80 + \text{код ошибки}$.
- Для команд чтения: $cmd = 0x81$, $status = 0x80 + \text{код ошибки}$.
- Поля *src* и *dst* меняются местами по сравнению с запросом.
- $byteCount = 0$ - нет данных.

7.2. Коды ошибок, дополнение к кодам протокола VDB

Код ошибки	Значение	Описание
0x85	$0x80 + 0x05$	Ошибка формата данных или выход за диапазон
0x86	$0x80 + 0x06$	Неверное количество данных
0x87	$0x80 + 0x07$	Команда недоступна в текущем режиме

8. Логика работы

8.1. Инициализация

1. Хост отправляет запрос версии протокола для проверки совместимости.
2. Устройство отвечает структурой *Version*.

8.2. Управление устройством

1. Хост отправляет команды записи для установки режима или управления позицией/скоростью.
2. В режиме *dutty* команды записи игнорируются.

8.3. Мониторинг состояния

1. Хост периодически запрашивает состояние устройства (*status*).
2. Устройство возвращает телеметрию в структуре *TelemetryUART*.

8.4. Обработка ошибок

1. При получении сообщения с ошибкой хост анализирует код ошибки.
2. В зависимости от кода ошибки хост может повторить команду или прекратить работу.

8.5. Таймауты и повторные попытки

- Таймаут ожидания ответа: 5 мс.
- Количество повторных попыток: 3.
- Интервал между попытками: 5 мс.
- Действие при отсутствии ответа: Физическое отключение питания устройства.

9. Алгоритм расчета контрольной суммы

Расчёт значения CRC8 производится для всех байт пакета, включая стартовый, на базе полинома $X^8 + X^5 + X^4 + 1$ (начальное значение 0xFF), XorOut = 0x00.

```
#define CRC_TABLE_SIZE 256
/** CRC polynome table */
uint8_t _CRC8Table[CRC_TABLE_SIZE];
/** Polynome width */ uint32_t _poly_width = 8;

/**
 * @brief init_crc_calculation
 * @param poly - полином
 * @param init_value - начальное значение
 * @param xor_value - значение XorOut
 * / void init_crc_calculation(uint8_t poly, uint8_t init_value, uint8_t xor_value) {
    _init_value = init_value;
    _xor_value = xor_value;

    // fill table
    const uint32_t bits_mask = (1 << _poly_width) - 1;
    const uint32_t top_bit = 1 << (_poly_width - 1);
    uint32_t index;
    for (index = 0; index < CRC_TABLE_SIZE; ++index) {
        uint32_t value = index << (_poly_width - 8);
        uint32_t bit_index;
        for (bit_index = 0; bit_index < 8; ++bit_index) {
            if (value & top_bit) {
                value = (value << 1) ^ poly;
            } else {
                value = value << 1;
            }
            value &= bits_mask;
        }
        _CRC8Table[index] = value;
    }
    _is_initied = 1;
}
```

```

/**
 * @brief crc_calc
 * @param data - данные
 * @param size - размер данных
 * @return
 * /
uint8_t crc_calc(uint8_t *data, uint8_t size) {
    uint8_t crc = _init_value;
    while (size--) {
        crc = _CRC8Table[crc л *data++];
    }
    return crc;
}

```

Для инициализации расчета CRC необходимо вызвать функцию инициализации со следующими аргументами: `init_crc_calculation(0x31, 0xFF, 0x00);`

10. Примеры сообщений

10.1. Запрос версии протокола

Запрос: [0xAA] [VDB: cmd=0x01, status=0x80, src=0x0000, dst=0x0100, byteCount=0]
[CRC8]

Ответ: [0xAA] [VDB: cmd=0x81, status=0x80, src=0x0100, dst=0x0000, byteCount=12, payload=Version] [CRC8]

10.2. Запрос состояния устройства

Запрос: [0xAA] [VDB: cmd=0x01, status=0x80, src=0x0000, dst=0x0501, byteCount=0]
[CRC8=0x15]

Ответ: [0xAA] [VDB: cmd=0x81, status=0x80, src=0x0501, dst=0x0000, byteCount=47, payload=TelemetryUART] [CRC8]